

УДК 519.688:004.4'2

А. Б. Глони́на¹

ИНСТРУМЕНТАЛЬНАЯ СИСТЕМА ПРОВЕРКИ ВЫПОЛНЕНИЯ ОГРАНИЧЕНИЙ РЕАЛЬНОГО ВРЕМЕНИ ДЛЯ КОНФИГУРАЦИЙ МОДУЛЬНЫХ ВЫЧИСЛИТЕЛЬНЫХ СИСТЕМ*

В работе рассматривается задача проверки выполнения ограничений реального времени для модульных вычислительных систем (МВС) на примере систем интегрированной модульной авионики. Сформулированы требования к программным средствам проверки выполнения этих ограничений. Описан подход к моделированию МВС, основанный на математическом аппарате сетей временных автоматов с остановкой таймеров и представлена инструментальная система, реализующая этот подход. Проведено экспериментальное исследование параметризованной модели МВС, входящей в состав разработанной инструментальной системы, а также ее апробация в составе используемой в отечественной промышленности САПР «Планировщик ИМА».

Ключевые слова: сети временных автоматов, интегрированная модульная авионика, планирование вычислений, верификация.

1. Введение. Модульная архитектура широко используется в современных и перспективных бортовых вычислительных системах. В данной работе модульные вычислительные системы (МВС) рассматриваются на примере систем интегрированной модульной авионики (ИМА).

Модульные вычислительные системы, в том числе системы ИМА, состоят из стандартизированных модулей, каждый из которых содержит один или несколько процессоров, как правило многоядерных. Взаимодействие между модулями осуществляется посредством коммутируемой сети с виртуальными каналами.

Рабочая нагрузка на процессорные ядра представлена набором прикладных задач. Все задачи являются периодическими: за период должен выполняться один экземпляр задачи, называемый работой. Задачи объединены в разделы — группы логически связанных прикладных задач, взаимодействующих посредством общей памяти. Как правило, один раздел соответствует одному приложению. Каждый раздел привязан к вычислительному ядру в составе процессора. Несколько разделов могут быть привязаны к одному и тому же ядру. На интервале планирования для вычислительного ядра задается статическое расписание окон, т.е. отрезков времени, в течение каждого из которых могут выполняться работы определенного раздела. В большинстве случаев интервал планирования, на котором строится расписание окон, равен наименьшему общему кратному периодов всех задач МВС.

В каждом разделе имеется собственный планировщик, управляющий постановкой на выполнение работ внутри окон раздела. Как правило, в системах ИМА используются динамические планировщики. Так стандарт ARINC 653 [1] предполагает использование динамического планировщика с фиксированными приоритетами и вытеснением. Допускается использование разных алгоритмов планирования в разных разделах. Для каждой задачи и каждого типа ядра известно наихудшее (максимальное) время выполнения одиночной работы данной задачи на ядре данного типа. В действительности время выполнения работ может быть меньше этого времени, однако на практике [2,3] при планировании вычислений принято считать, что для каждой работы длительность выполнения на ядре фиксирована и равна заданной величине. У каждой работы существует директивный интервал, определяющийся периодом задачи, номером работы, а также смещениями левой и правой границ этого интервала относительно начала периода. При этом для задачи,

¹ Факультет ВМК МГУ, программист, e-mail: alevtina@lvk.cs.msu.su

* Работа выполнена при поддержке РФФИ, проект № 19-07-00614.

имеющей период p , директивный интервал i -й работы, $i \geq 1$, принадлежит отрезку $[(i-1) \cdot p, i \cdot p]$. Работа становится готовой к выполнению не ранее определенной для нее левой границы директивного интервала. Работа должна завершиться не позднее правой границы своего директивного интервала. Если правая граница директивного интервала достигается до завершения выполнения работы, то работа считается *опоздавшей* и не может далее выполняться на вычислительном ядре. В таком случае время, которое опоздавшая работа находилась на вычислительном ядре, меньше заданного времени, необходимого для ее выполнения.

Между задачами с одинаковым периодом могут существовать зависимости по данным: очередная работа задачи-получателя считается не готовой к выполнению до тех пор, пока не получит данные от всех соответствующих работ задач-отправителей, а также пока текущее время не достигнет левой границы директивного интервала этой работы. Данные передаются в сообщениях фиксированного (для каждой пары задач) размера. Для каждого сообщения известны верхние оценки длительности передачи через внутреннюю память модуля и через сеть. По аналогии с длительностями выполнения работ, при планировании вычислений принято считать, что для каждого сообщения длительность передачи фиксирована и равна одной из двух (в зависимости от того, привязаны ли отправитель и получатель к одному модулю) указанных оценок.

Конфигурация МВС определяет состав и характеристики процессорных ресурсов с их разделением по модулям, состав и характеристики рабочей нагрузки, ее распределение по ядрам, а также расписания окон для каждого ядра.

2. Задача проверки выполнения ограничений реального времени для конфигураций МВС и требования к методам ее решения. Для конфигурации МВС выполняются ограничения реального времени, если для нее в процессе функционирования ни одна из работ не становится опоздавшей, т.е. для каждой работы длительность ее выполнения на вычислительном ядре равна заданному в конфигурации значению. Это условие будем называть *критерием выполнения ограничений реального времени*.

При проектировании МВС возникают задачи выбора наилучшей по некоторому критерию конфигурации, удовлетворяющей ограничениям реального времени. Примерами минимизируемых критериев оптимизации являются нагрузка на среду передачи данных [2, 4], число переключений контекста [3, 5], стоимость [6, 7], масса [6, 7] и энергопотребление [7] системы. Также известны задачи поиска конфигурации, удовлетворяющей ограничениям реального времени, среди множества потенциальных конфигураций без выделения оптимизируемого критерия [8–10].

Одним из широко используемых на практике методов проверки ограничений реального времени для МВС с заданной конфигурацией является построение *временной диаграммы* (ВД) функционирования этой МВС при длительностях выполнения прикладных задач и обменов данными, равных заданным верхним оценкам. Временная диаграмма должна содержать интервалы выполнения работ. Получив такую ВД, можно непосредственно проверить выполнение ограничений реального времени. Временная диаграмма функционирования МВС с заданной конфигурацией может быть построена посредством прогона имитационной модели такой МВС.

По результатам анализа работ, в которых решаются задачи, требующие проверки выполнения ограничений реального времени для конфигураций МВС [2–10], автором были сформулированы следующие требования к средствам моделирования МВС.

1. *Возможность получения ВД, включающей события постановки на выполнение, события вытеснения и завершения работ.* Такой ВД достаточно для проверки выполнения ограничений реального времени для конфигурации МВС.
2. *Возможность моделирования тех аспектов функционирования МВС, которые существенны для проверки критерия выполнения ограничений реального времени.* В частности необходима возможность моделирования вытеснения работ, зависимостей по данным, использования различных алгоритмов планирования в различных разделах, использования нескольких типов вычислительных ядер, различающихся производительностью.

3. *Возможность формальной проверки корректности моделей.* Такая проверка необходима, так как на этапе проектирования МВС невозможно экспериментально сравнить поведение модели с поведением реальной МВС.
4. *Поддержка библиотеки моделей компонентов МВС.* Различные МВС строятся из стандартизированных компонентов, поэтому и модели МВС целесообразно строить из моделей этих компонентов, образующих библиотеку моделей.
5. *Автоматизация построения и прогона модели.* Для того чтобы система моделирования могла использоваться в цикле работы оптимизационных алгоритмов, процесс построения модели по описанию конфигурации, а также прогона модели и анализа результатов моделирования должен быть полностью автоматизирован.
6. *Возможность расширения библиотеки моделей компонентов МВС.* В процессе проектирования может потребоваться анализ конфигураций, включающих компоненты (планировщики, среды передачи данных и т.п.), моделей которых нет в библиотеке. Для этого у проектировщика должна быть возможность самостоятельно создать новые модели компонентов системы.
7. *Скорость построения и прогона модели, а также анализа результатов моделирования,* должна позволять работать с конфигурациями размерности, соответствующей реальным системам, с учетом числа конфигураций, оцениваемых при проектировании (это число может достигать нескольких сотен и более при работе оптимизационных алгоритмов).

Автором был проведен обзор ряда средств моделирования МВС, в результате которого не было выявлено ни одного средства, в полной мере удовлетворяющего перечисленным требованиям. Так CHEDDAR [11] поддерживает наличие лишь одного окна на интервале планирования, HSSim [12] поддерживает ограниченный набор алгоритмов планирования и не позволяет задать статическое расписание окон, ДИАНА и ДИАНА-2012 [13,14] не позволяют моделировать вытеснение, инструменты, описанные в работах [2,8], поддерживают лишь один алгоритм планирования, средство MASIW [15] ориентировано на строго периодическое выполнение прикладных задач (т.е. время постановки на выполнение каждой работы строго равно левой границе ее директивного интервала), модель, описанная в [16], не поддерживает зависимости по данным. Таким образом, перечисленные средства не удовлетворяют требованию 2. Средства, приведенные в [2,8,12,16,17], не позволяют формально проверять корректность моделей, т.е. не удовлетворяют требованию 3. Средства SMART [17] и MASIW [15] имеют только графический интерфейс и, следовательно, не удовлетворяют требованию 5. Предложенные в [2,8,12,16] средства не удовлетворяют требованию 6, т.е. не позволяют использовать пользовательские модели компонентов МВС. Анализ тех средств, исходный код которых находится в открытом доступе [11,12,14,15], показал, что их доработка для поддержки всех перечисленных требований значительно более трудоемка, чем создание нового средства.

3. Предложенный подход к проверке выполнения ограничений реального времени.

3.1. Математический аппарат для моделирования функционирования МВС. В качестве математического аппарата для моделирования функционирования МВС был выбран математический аппарат сетей временных автоматов с остановкой таймеров [18], так как он соответствует требованиям 2 и 3 п. 2, т.е. позволяет моделировать все существенные для проверки выполнения ограничений реального времени аспекты функционирования МВС, а также формально проверять корректность моделей.

Временной автомат с остановкой таймеров представляет собой конечный автомат с целочисленными переменными и таймерами. Далее для обозначения временного автомата с остановкой таймеров используется термин “автомат”. Таймер — это специальная переменная, принимающая

вещественные значения. Таймер считается активным, если его условие активности (булево выражение над множеством переменных) в текущей локации автомата истинно. В противном случае таймер считается остановленным.

Состоянием автомата называется совокупность его текущей локации, значений переменных и таймеров. Состояние автомата может измениться в результате выполнения перехода либо в результате увеличения значений всех активных таймеров на одно и то же значение.

Сеть автоматов представляет собой набор автоматов, функционирующих совместно и взаимодействующих при помощи общих переменных и синхронизаций по каналам. Синхронизации автоматов выполняются посредством приема и отправки сообщений по каналам. Если в некоторый момент времени в одном из автоматов сети может быть выполнен переход, помеченный отправкой сигнала по каналу, а в другом автомате сети — приемом сигнала по тому же каналу, то оба перехода выполняются как единое действие.

Состояние сети автоматов определяется совокупностью состояний ее автоматов. Вычислением сети автоматов является всякая возможная конечная или бесконечная последовательность ее состояний.

Для построения модели МВС, соответствующей перечисленным в п. 2 требованиям, в математический аппарат сетей временных автоматов с остановкой таймеров были добавлены понятия и сущности, приведенные ниже и выделенные курсивом.

Набор переменных и действий синхронизации, посредством которых некоторый автомат взаимодействует с другими автоматами сети, называется *интерфейсом этого автомата*.

Модельное время сети автоматов — значение некоторого служебного таймера, условие активности которого всегда истинно и который никогда не обнуляется.

Событие синхронизации автоматов — это тройка $e = \langle CH, A, t \rangle$, соответствующая содержащему синхронизацию переходу в сети автоматов, где CH — канал синхронизации, A — набор автоматов, участвующих в синхронизации, t — модельное время сети автоматов.

Временная диаграмма (ВД) сети автоматов — набор событий синхронизации, соответствующий некоторому вычислению данной сети. Для того чтобы по заданному вычислению сети автоматов получить ВД сети автоматов, необходимо выбрать из этого вычисления все переходы, содержащие синхронизации, и каждому переходу поставить в соответствие событие синхронизации.

Параметризованный автомат — это автомат, в арифметических и логических выражениях которого наряду с числами и переменными используются целочисленные параметры с незадаанными значениями. *Экземпляр автомата* порождается по параметризованному автомату путем задания константных числовых значений для параметров этого параметризованного автомата.

Базовый тип автоматов определяется только интерфейсом.

Параметризованный автомат *реализует* базовый тип автоматов, если интерфейс параметризованного автомата содержит все переменные и действия синхронизации интерфейса базового типа, а также, возможно, другие переменные и действия синхронизации.

Набор базовых типов автоматов называется *обобщенной сетью автоматов*. Набор параметризованных автоматов — *параметризованной сетью автоматов*, сеть экземпляров автоматов — *экземпляром сети автоматов*.

Введенные уровни абстракции автоматов согласуются в рамках предлагаемой схемы моделирования с уровнями абстракции компонентов МВС. Базовому типу автоматов соответствует базовый тип компонентов. Базовый тип компонента МВС определяет сервис, предоставляемый компонентом системы, но не логику его функционирования. Примером базового типа компонента является планировщик раздела. Параметризованному автомату соответствует конкретный тип компонента. Конкретный тип компонента МВС определяет детали реализации базового типа. Примером конкретного типа компонента является планировщик раздела, работающий по алгоритму планирования с фиксированными приоритетами и вытеснением (FPPS). Экземпляр автомата соответствует экземпляру компонента системы.

3.2. Обобщенная модель МВС и построение моделей конкретных МВС на ее основе. Предложено моделировать МВС на трех введенных уровнях абстракции:

1. *Обобщенная модель МВС* — это обобщенная сеть автоматов, состоящая из следующих базовых типов автоматов:

- базовый тип автоматов **T**, моделирующий функциональную задачу;
- базовый тип автоматов **TS**, моделирующий планировщик работ раздела;
- базовый тип автоматов **CS**, моделирующий планировщик ядра;
- базовый тип автоматов **L**, моделирующий виртуальный канал. Под виртуальным каналом понимается средство передачи сообщений в общем случае.

Для каждого базового типа автоматов описан его интерфейс. Так, например, автоматы базового типа **CS** отправляют сигналы по каналам **wakeup_i** и **sleep_i**, где i -й канал соответствует i -му разделу.

2. *Параметризованная модель МВС* — это параметризованная сеть автоматов, состоящая из автоматов, реализующих перечисленные базовые типы. При этом для каждого базового типа автомата в параметризованной модели должен присутствовать хотя бы один параметризованный автомат, реализующий этот базовый тип. Параметризованная модель МВС содержит следующие разработанные автором параметризованные автоматы, а именно модели: задачи, планировщика ядра, виртуального канала и нескольких планировщиков работ, работающих согласно различным алгоритмам планирования (с фиксированными и динамическими приоритетами, с вытеснением и без вытеснения). В параметризованную модель МВС могут быть добавлены новые (пользовательские) модели, реализующие тот или иной базовый тип автоматов из обобщенной модели.

3. *Модель МВС с заданной конфигурацией* — это экземпляр сети автоматов, полученный на основе параметризованной модели МВС по описанию конфигурации. Каждому компоненту МВС, описанному в конфигурации (функциональной задаче, разделу, вычислительному ядру, сообщению), однозначно соответствует экземпляр автомата (модель этого компонента). Параметры автоматов устанавливаются в соответствующие значения из описания конфигурации.

По ВД модели МВС с заданной конфигурацией можно однозначным образом получить ВД этой МВС, достаточную для проверки выполнения ограничений реального времени. Таким образом, выполняется требование 1 п. 2.

Для сложности построения и прогона модели МВС с заданной конфигурацией были получены следующие оценки сверху соответственно: $O(N + W + T + H)$, $O(L \cdot (T^2 + H^2))$, где N — количество вычислительных ядер, W — количество окон, T — количество функциональных задач, H — количество сообщений, L — длительность интервала планирования.

Подробное описание обобщенной модели МВС, а также метода построения на ее основе модели МВС с заданной конфигурацией приведено в [19].

3.3. Обоснование корректности формируемых моделей. Согласно предположениям, используемым при проектировании МВС на этапе выбора конфигурации (например, в работах [2, 3, 8]), каждой конфигурации однозначно соответствует ВД МВС. Поэтому модель МВС с заданной конфигурацией должна быть детерминированной, т.е. временные диаграммы этой модели (сети автоматов), построенные по всем возможным ее вычислениям, должны совпадать.

Корректность модели МВС вводится как соответствие всех ВД этой модели набору требований, представляющих собой ограничения на последовательность событий и длительность интервалов между событиями. Эти требования были выделены из стандартов на МВС, описаний схем планирования, работ по планированию вычислений в МВС.

Подход к обоснованию корректности всего класса моделей, синтезируемых согласно предложенному методу, описан в работе [20]. Согласно этому подходу, для каждого базового типа автомата, входящего в состав обобщенной модели МВС, был выделен набор требований, которым должны удовлетворять все параметризованные автоматы, реализующие данный базовый

тип. Было доказано, что если для всех параметризованных автоматов, формирующих параметризованную модель МВС, выполняются требования, сформулированные для соответствующих базовых типов автоматов, то все модели МВС, построенные на основе этой параметризованной модели, удовлетворяют всем выделенным требованиям корректности к модели в целом, а также являются детерминированными.

Для построенной параметризованной модели МВС было проверено выполнение требований к формирующим ее параметризованным автоматам. Проверка выполнения требований к каждому параметризованному автомату проводится автоматически с помощью верификатора. Для этого используется подход автоматов-наблюдателей [21]. Автомат-наблюдатель соответствует одному требованию корректности и представляет собой автомат, имеющий интерфейс для взаимодействия с исходным автоматом и функционирующий в композиции с исходным автоматом. Автомат-наблюдатель имеет некоторую “плохую” локацию, такую, что любая не удовлетворяющая требованию последовательность синхронизаций гарантированно приводит автомат-наблюдатель в “плохую” локацию. Таким образом, проверка выполнения требования сводится к проверке недостижимости этой локации. Приведем пример требования корректности, выделенного из стандарта ARINC 653 [1]:

“Для любого раздела верно, что в каждый момент времени на вычислителе может выполняться не более одной работы этого раздела”.

Чтобы при добавлении в параметризованную модель МВС нового параметризованного автомата полученная параметризованная модель была корректной, достаточно убедиться (с использованием верификатора), что все требования к соответствующему базовому типу автомата для данного автомата выполняются.

4. Инструментальная система проверки выполнения ограничений реального времени.

4.1. Состав и схема работы инструментальной системы. Для исследования и апробации предложенных методов была разработана реализующая их инструментальная система. Эта система используется для получения параметризованной модели МВС в виде исполняемого файла, т.е. параметризованной программной модели МВС (см. рис. 1), и для проверки выполнения ограничений реального времени для МВС с заданной конфигурацией с помощью полученной модели (см. рис. 2).

В состав разработанной инструментальной системы входят следующие компоненты (инструменты и создаваемые с их помощью объекты), приведенные на рис. 1 и 2.

1. *UPPAAL* — инструмент построения и верификации временных автоматов с остановкой таймеров, разработанный в университете г. Уппсала, Швеция [22]. В рамках настоящей работы графический редактор UPPAAL используется для создания, модификации и просмотра автоматов (как автоматов-моделей компонентов МВС, так и автоматов-наблюдателей для проверки выполнения требований корректности к моделям). Верификатор UPPAAL используется для проверки выполнения требований корректности к моделям компонентов МВС, т.е. для проверки недостижимости “плохих” локаций в автоматах-наблюдателях, верифицируемых совместно с автоматами-моделями.

2. *Автоматная модель компонента МВС* — параметризованный автомат, моделирующий компонент МВС и представленный в формате средства UPPAAL.

3. *Требования корректности к моделям компонентов МВС* представлены в виде набора автоматов-наблюдателей, описанных в формате средства UPPAAL. Для каждого базового типа автоматов сформулированы требования корректности, которым должны удовлетворять все параметризованные автоматы, реализующие этот базовый тип. Также определены требования, специфичные для построенных моделей конкретных типов компонентов МВС. Предложенный автором подход к обоснованию корректности моделей описан в работе [20]. При разработке новых моделей компонентов МВС набор требований корректности может быть расширен.

4. *Библиотека автоматных моделей компонентов МВС* — набор автоматных моделей компонентов МВС. Для каждой модели этого набора выполнены все соответствующие требования

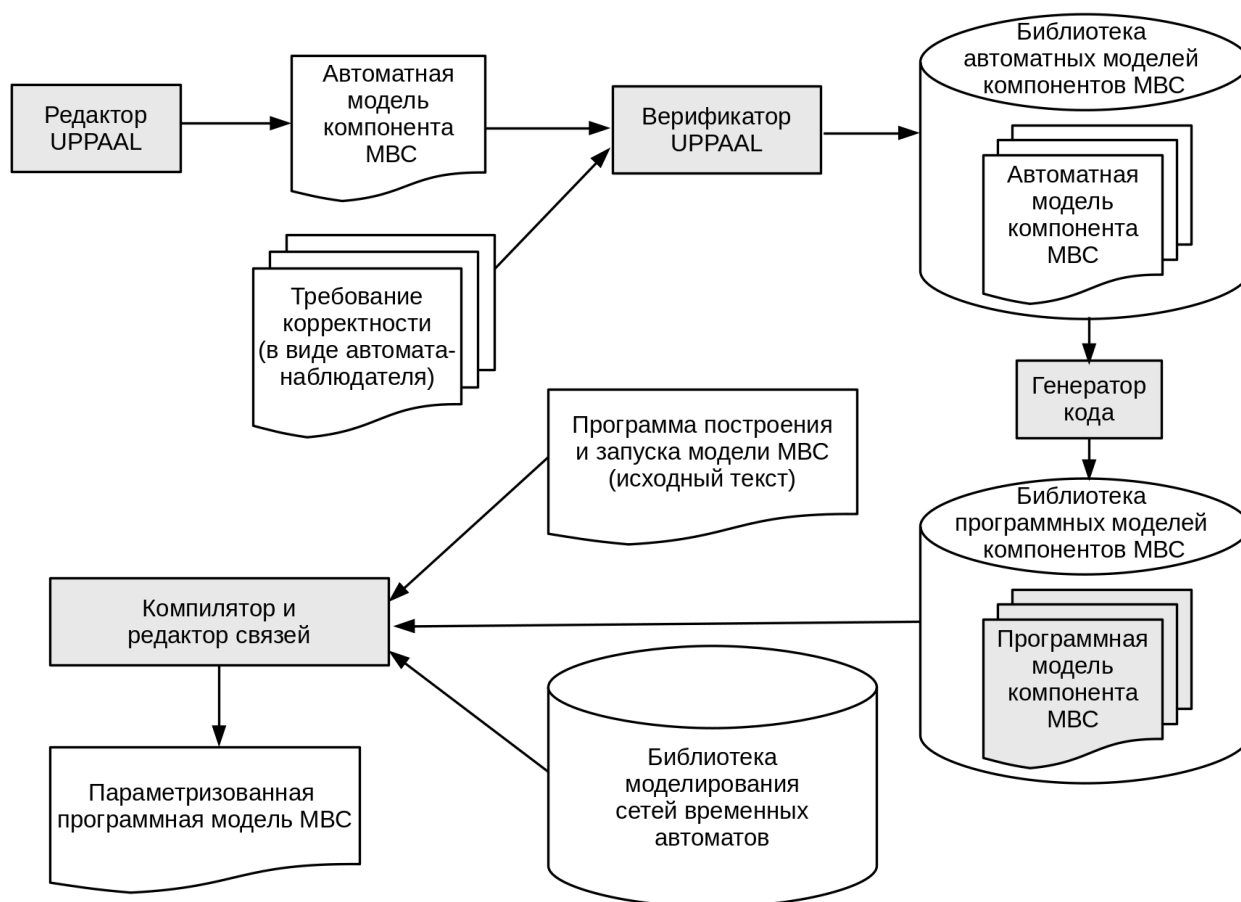


Рис. 1. Схема получения параметризованной программной модели MCS

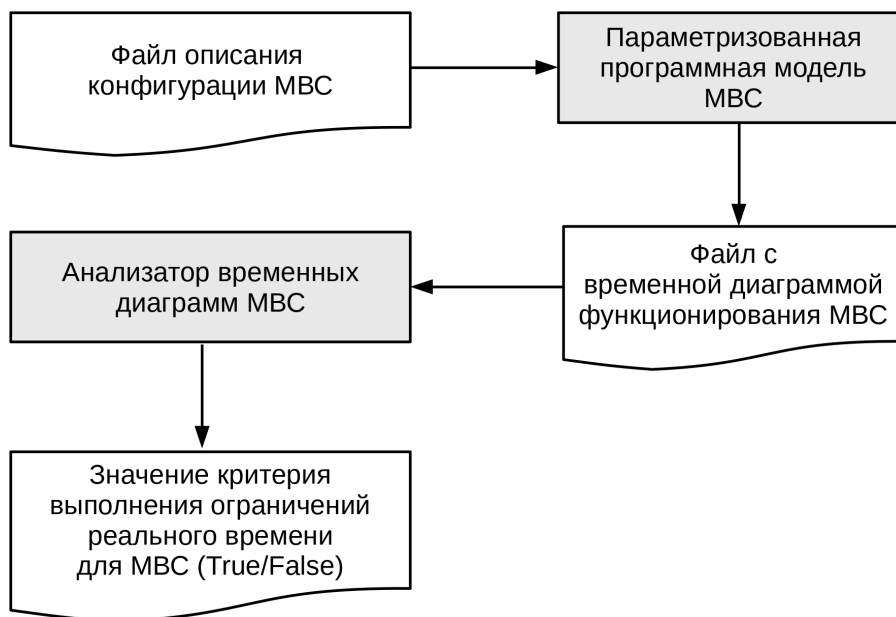


Рис. 2. Схема использования параметризованной программной модели MCS

корректности. Данная библиотека содержит разработанные автором модели планировщика ядра, функциональной задачи, виртуального канала, а также трех часто используемых в MCS планировщиков работ раздела: с фиксированными приоритетами и вытеснением; с фиксированным приоритетом без вытеснения; планировщик, работающий по стратегии EDF с вытеснением. Библиотека может быть пополнена новыми моделями. Для этого необходимо с использованием

верификатора UPPAAL проверить, что для новых моделей выполнены соответствующие требования (как минимум — требования для базовых типов компонентов МВС).

5. *Библиотека моделирования сетей временных автоматов* — разработанная автором библиотека на языке C++, содержащая программные реализации всех элементов математического аппарата сетей временных автоматов с остановкой таймеров и используемая для получения вычисления сети автоматов, моделирующей МВС. В библиотеке имеются классы, соответствующие автомату, локациям, переходу, таймеру и т.д. В открытых источниках не было найдено средства, позволяющего автоматически получать вычисления сетей автоматов с остановкой таймеров. В частности, UPPAAL предоставляет для этого лишь графический интерфейс, требующий ручных действий, что не соответствует требованию 5 п. 2. Поэтому было принято решение о разработке собственного средства моделирования сетей временных автоматов.

6. *Программная модель компонента МВС* — код на языке C++, моделирующий компонент МВС и являющийся C++-представлением параметризованного автомата-модели этого компонента. В данном коде используются классы, реализованные в библиотеке моделирования сетей временных автоматов. Программная модель компонента МВС формируется автоматически по соответствующей автоматной модели с использованием генератора кода.

7. *Генератор кода* — разработанный автором инструмент, позволяющий сформировать для параметризованного автомата, описанного в формате UPPAAL, его представление на языке C++. Генератор необходим для автоматического получения по автоматным моделям компонентов МВС соответствующих программных моделей, так как выполнение такой трансляции вручную требует существенных трудозатрат и, кроме того, может привести к внесению ошибок в модели. Данный инструмент написан на языке Python с использованием средства генерации текста по шаблонам Cheetah [23]. Сгенерированный код содержит объекты классов библиотеки моделирования сетей временных автоматов, а также классы-наследники библиотечных классов и объекты этих классов-наследников. Кроме того, в сгенерированном коде имеются функции, каждая из которых соответствует одному базовому типу автоматов. Такая функция принимает на вход имя параметризованного автомата, реализующего базовый тип, и создает объект класса, реализующего этот параметризованный автомат. Наличие таких функций дает возможность не модифицировать код построения экземпляра модели МВС по описанию конфигурации при появлении новых моделей планировщиков разделов: для порождения объекта соответствующего класса достаточно указать имя модели планировщика в описании конфигурации.

8. *Библиотека программных моделей компонентов МВС* — библиотека на языке C++, полученная автором с помощью генератора кода по автоматам из библиотеки автоматных моделей компонентов МВС. Для того, чтобы добавить в библиотеку новую модель, необходимо добавить в библиотеку автоматных моделей соответствующий параметризованный автомат, реализующий один из базовых типов автоматов, а затем запустить для обновленной библиотеки автоматных моделей генератор кода.

9. *Компилятор и редактор связей* — стандартные инструменты из набора GCC для получения исполняемого кода программы из ее исходных текстов с использованием библиотек. Эти инструменты необходимы для получения параметризованной программной модели МВС, использующей библиотеку моделирования сетей временных автоматов и библиотеку программных моделей компонентов МВС.

10. *Программа построения и запуска модели МВС (исходный текст)* — разработанная автором программа на языке C++, реализующая последовательное выполнение трех этапов:

- построение модели МВС по описанию конфигурации системы. Формальное описание алгоритма построения приведено в работе [19];
- прогон построенной модели, в процессе которого формируется ВД сети автоматов;
- построение по ВД сети автоматов искомой ВД МВС.

11. *Параметризованная программная модель МВС* — исполняемый файл, получаемый в результате компиляции и сборки исходного кода программы построения и запуска модели МВС с

использованием библиотеки программных моделей компонентов МВС и библиотеки моделирования сетей временных автоматов.

12. *Файл описания конфигурации МВС* — файл в формате XML, содержащий информацию о количестве, типах, числовых характеристиках и связях компонентов системы. Используется параметризованной программной моделью МВС для построения модели конкретной МВС, а также анализатором ВД для проверки выполнения ограничений реального времени для конфигурации МВС.

13. *Файл с ВД МВС* — файл в формате XML с данными о событиях постановки на выполнение, событиях вытеснения и завершения работ. Используется анализатором ВД для проверки выполнения ограничений реального времени для конфигурации МВС.

14. *Анализатор ВД МВС* — разработанная автором программа на языке C++, проверяющая выполнение ограничений реального времени для МВС с заданной конфигурацией в соответствии с критерием, сформулированным в п. 2. Результатом работы программы является значение критерия (булево значение True или False).

В соответствии со схемой, приведенной на рис. 1, автоматные модели компонентов МВС создаются в редакторе UPPAAL [22]. Далее, с помощью верификатора UPPAAL для каждой разработанной модели проверяется выполнение требований корректности, сформулированных для соответствующего базового типа автоматов, а также, возможно, дополнительных требований, специфичных для этой модели. Таким образом, выполняется требование 4 п. 2.

Автоматные модели компонентов МВС, для которых выполнены все необходимые требования, формируют библиотеку автоматных моделей компонентов МВС. Для каждой автоматной модели с помощью генератора кода формируется соответствующая ей программная модель. Полученные программные модели формируют библиотеку программных моделей компонентов МВС.

Далее посредством компиляции и сборки на основе библиотеки программных моделей компонентов МВС, библиотеки моделирования сетей временных автоматов и программы построения и запуска модели МВС получается исполняемая параметризованная программная модель МВС.

Важной особенностью разработанной инструментальной системы является то, что после добавления в библиотеку автоматных моделей новой модели компонента МВС для использования этой модели достаточно запуска генератора кода и пересборки исполняемой параметризованной программной модели МВС. Модификации кода этой модели не требуется. Следовательно, выполняется требование 6 п. 2. Для фиксированного состава библиотеки программных моделей компонентов МВС моделирование МВС различных конфигураций не требует пересборки исполняемой параметризованной программной модели, что позволяет сократить временные затраты при анализе большого числа конфигураций.

В соответствии со схемой, приведенной на рис. 2, для проверки выполнения ограничений реального времени для некоторой конфигурации МВС необходимо сформировать файл в формате XML с описанием этой конфигурации. Этот файл подается на вход параметризованной программной модели МВС. Результатом прогона этой модели является файл в формате XML, содержащий описание ВД МВС. Далее ВД МВС обрабатывается Анализатором ВД МВС, который проверяет критерий выполнения ограничений реального времени. Этап проверки этого критерия может выполняться другими, отличными от разработанного автором Анализатора, средствами. Например, при интеграции с САПР “Планировщик ИМА” (см. п. 4.2) проверка выполнения ограничений реального времени по полученной ВД выполняется средствами самой САПР. В соответствии с описанной схемой построение и прогон модели МВС с заданной конфигурацией выполняется автоматически и, следовательно, выполняется требование 5 п. 2.

Таким образом, разработанная инструментальная система удовлетворяет требованиям 1–6 п. 2. Выполнение требования 7 проверено экспериментально (результаты приведены в п. 5).

4.2. Интеграция с САПР “Планировщик ИМА”. Для апробации разработанной инструментальной системы была выполнена ее интеграция с используемой в отечественной промышленности САПР “Планировщик ИМА” [2].

Система автоматизированного проектирования “Планировщик ИМА” используется при про-

ектировании МВС с архитектурой ИМА и предназначена для решения задачи распределения рабочей нагрузки по вычислительным ядрам, задачи построения окон разделов и назначения приоритетов функциональных задач.

На вход САПР подается описание МВС с архитектурой ИМА, содержащее следующие характеристики:

- набор модулей;
- набор типов используемых процессоров;
- для каждого модуля — число и тип процессоров;
- для каждого процессора — число ядер, время инициализации окна и переключения контекста между разделами;
- для каждого ядра — наибольшее допустимое значение загрузки.

Также задается описание рабочей нагрузки:

- набор задач; для каждой задачи — период, начальное значение приоритета (начальные значения приоритетов нескольких задач одного раздела могут совпадать), время выполнения на каждом из типов процессоров;
- набор разделов; для каждого раздела — набор принадлежащих ему задач и набор ядер, к которым может быть привязан данный раздел;
- набор сообщений; для каждого сообщения — задача-отправитель, задача-получатель, размер и максимальные задержки на передачу через бортовую сеть и через память модуля.

Выполнением работ разделов управляет планировщик с фиксированными приоритетами и вытеснением. Считается, что левая граница директивного интервала очередной работы задачи равна началу соответствующего периода, а правая граница директивного интервала — окончанию соответствующего периода.

Необходимо найти привязку разделов к ядрам, расписание окон разделов для каждого ядра и уточненные значения приоритетов задач, уникальные в рамках раздела. При этом объем сообщений, передаваемых через бортовую сеть в течение интервала планирования, должен быть минимальным, и найденная конфигурация должна удовлетворять критерию выполнения ограничений реального времени.

Для решения описанной задачи в САПР “Планировщик ИМА” используются различные оптимизационные алгоритмы (жадный [2], генетический [24], метод ветвей и границ [25]). В процессе работы алгоритма оптимизации формируется некоторая конфигурация, для которой требуется проверить критерий выполнения ограничений реального времени. Если ограничения реального времени выполнены, конфигурация может быть принята в качестве решения задачи. В противном случае конфигурация отбрасывается и поиск продолжается.

В САПР “Планировщик ИМА” существует встроенная имитационная модель для проверки выполнения ограничений реального времени. Однако для нее не имеется формального обоснования корректности, и в ее основе нет математического аппарата, который позволил бы такое обоснование выполнить. Поэтому целесообразно заменить ее на параметризованную программную модель МВС, формируемую с помощью разработанной автором инструментальной системы и не обладающую указанными недостатками.

Схема интеграции разработанной инструментальной системы с САПР “Планировщик ИМА” представлена на рис. 3. Из состава инструментальной системы на данной схеме приведена лишь параметризованная программная модель МВС, а также файлы с ее входными и выходными данными. Остальные компоненты инструментальной системы требуются для формирования этой модели и в цикле работы САПР “Планировщик ИМА” не используются.

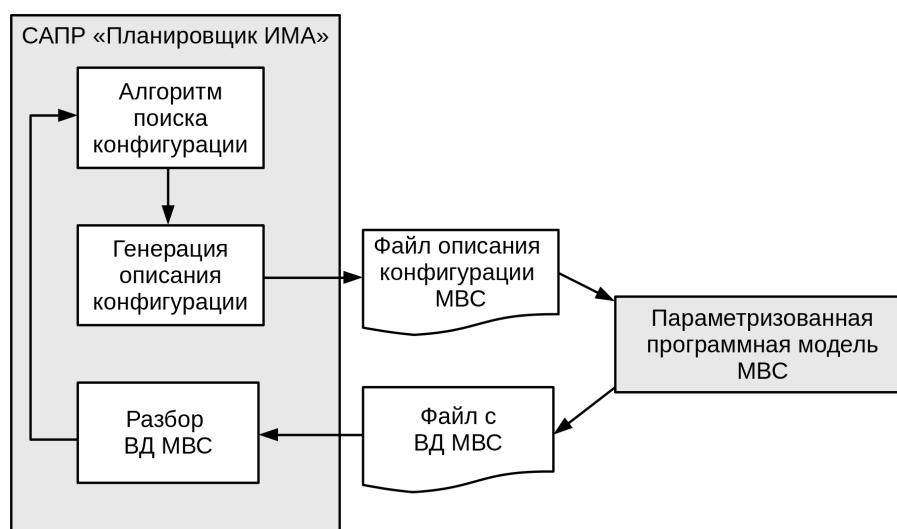


Рис. 3. Схема интеграции разработанной инструментальной системы с САПР «Планировщик ИМА»

На этапе работы поискового алгоритма, требующем проверки выполнения ограничений реального времени для некоторой конфигурации, генерируется файл описания этой конфигурации. Затем запускается параметризованная программная модель МВС, принимающая на вход сгенерированный файл. Результатом работы модели является файл с ВД МВС. Этот файл разбирается в САПР «Планировщик ИМА», и на основе его данных осуществляется проверка выполнения ограничений реального времени.

Для реализации этой схемы в САПР «Планировщик ИМА» были добавлены функции генерации файлов описаний конфигураций и разбора файлов с ВД МВС.

В отличие от общей схемы использования параметризованной программной модели МВС (см. рис. 2), при интеграции с САПР «Планировщик ИМА» не используется Анализатор ВД, а его функцию выполняет непосредственно САПР. Причиной этого является то, что САПР предоставляет пользователю не только значение критерия выполнения ограничений реального времени для МВС с заданной конфигурацией, но и дополнительные данные о функционировании этой МВС: загрузка ядер, информация об опоздавших работах, графическое представление ВД. Для получения этих данных необходима доступность всей ВД МВС в САПР.

5. Экспериментальное исследование. Экспериментальное исследование разработанных методов и средств имело следующие цели:

- сравнение с теоретическими оценками порядка зависимости времени построения и прогона модели МВС с заданной конфигурацией от характеристик конфигурации;
- оценка производительности и масштабируемости разработанных методов и средств, а также подтверждение их пригодности для анализа систем реальной размерности;
- сравнение по производительности разработанных методов и средств с моделью, встроенной в САПР «Планировщик ИМА» [2].

В качестве базовых исходных данных для проведения экспериментального исследования было выбрано описание МВС, соответствующее одной из современных авиационных бортовых систем с архитектурой ИМА: порядка 150 периодических задач, сгруппированных в 10 разделов, около 100 сообщений, 6 вычислительных ядер [2]. Конфигурация МВС, соответствующая этой системе и содержащая порядка 8000 окон на интервале планирования длительностью в 1 с (типовые значения, используемые в САПР «Планировщик ИМА»), будет далее обозначаться как $CONF_{base}$.

На основе этого описания было сформировано несколько наборов данных, использование которых позволило оценить зависимость времени построения и прогона модели от числа функциональных задач, сообщений, разделов, ядер, окон на интервале планирования. Большая часть

наборов данных была сгенерирована таким образом, чтобы в серии экспериментов, проводящейся для исследования зависимости времени работы разработанного средства от некоторой характеристики конфигурации, варьировалось значение этой характеристики, а остальные характеристики конфигурации оставались неизменными. Если изменение одной характеристики конфигурации МВС невозможно без изменения другой характеристики конфигурации МВС, то значения таких характеристик варьируются в серии экспериментов согласованно. Кроме того, в одной из серий экспериментов согласованно увеличивались все перечисленные характеристики. Наименьшее значение исследуемой характеристики в серии экспериментов как правило было равно ее значению в конфигурации $CONF_{base}$, наибольшее — ее значению в конфигурации $CONF_{base}$, увеличенному в 10 раз. Все эксперименты проводились на машине с процессором Intel(R) Xeon(R) CPU с тактовой частотой 2.30 ГГц и оперативной памятью размером 7.2 Гб. При этом объем оперативной памяти, используемой параметризованной программной моделью МВС, во всех экспериментах не превосходил 50 Кб.

Экспериментальное исследование показало, что для каждой характеристики конфигурации МВС порядок зависимости времени построения и прогона модели МВС от значения этой характеристики соответствует своей теоретической оценке.

Для конфигурации размерности, соответствующей реальной МВС ($CONF_{base}$), время проверки ограничений реального времени составило менее 2 с (на используемой для экспериментов машине), что позволяет за сутки обработать несколько десятков тысяч конфигураций. Следовательно, для конфигураций такой размерности разработанное средство может быть использовано в цикле работы поисковых алгоритмов, перебирающих большое число потенциальных конфигураций (например, эволюционных алгоритмов).

Результаты сравнения по производительности разработанной параметризованной модели МВС с моделью, встроенной в САПР «Планировщик ИМА», приведены на рис. 4. Эксперименты показали, что на конфигурациях небольшой размерности производительности этих двух средств приблизительно совпадают. Однако, с ростом размерности конфигураций разработанная модель начинает превосходить встроенную модель: чем больше размерность, тем больше превосходство.

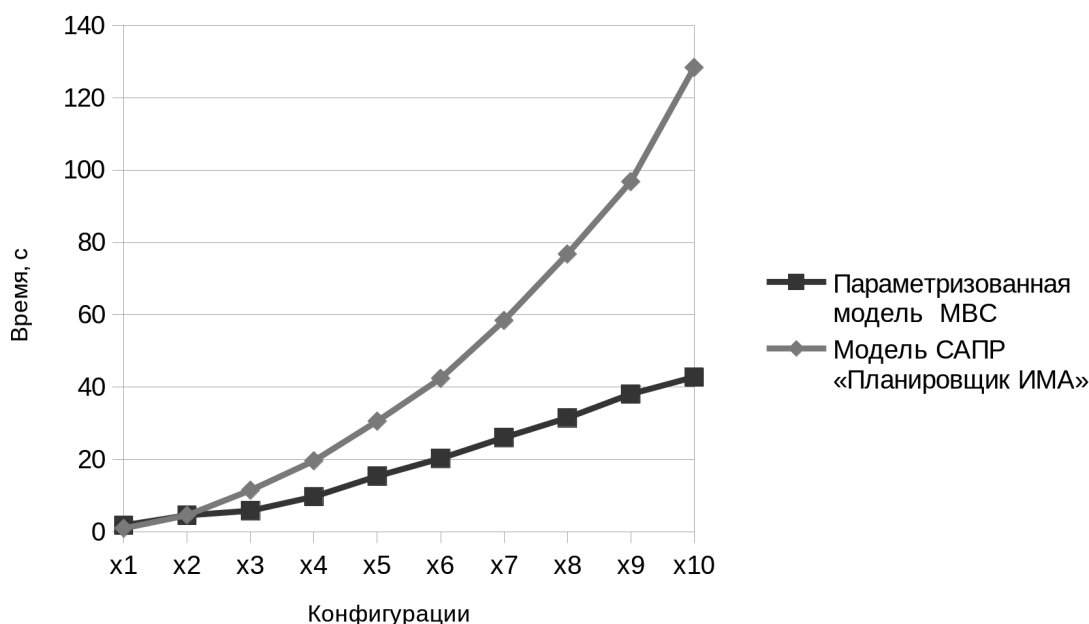


Рис. 4. Сравнение разработанной параметризованной модели МВС со встроенной в САПР «Планировщик ИМА» моделью по времени проверки ограничений реального времени

6. Заключение. Разработанная инструментальная система проверки выполнения ограничений реального времени для МВС удовлетворяет всем требованиям, сформулированным для

таких систем по результатам анализа задач проектирования МВС. В частности, инструментальная система позволяет автоматически формировать модель конкретной МВС по описанию ее конфигурации, в том числе с использованием пользовательских моделей компонентов системы. Выбранный для моделирования математический аппарат позволяет формально доказывать корректность формируемых моделей МВС. Для моделей, формируемых на основе разработанной библиотеки моделей компонентов МВС, такое доказательство было выполнено [20]. Экспериментальное исследование подтвердило применимость разработанных методов и средств для анализа конфигураций размерности, соответствующей реальным МВС. Инструментальная система успешно прошла апробацию в составе используемой в отечественной промышленности САПР «Планировщик ИМА».

СПИСОК ЛИТЕРАТУРЫ

1. Avionics Application software standard interface. ARINC specification 653. Annapolis: Aeronautical Radio, 1997.
2. Balashov V. V., Balakhanov V. A., Kostenko V. A. Scheduling of computational tasks in switched network-based IMA systems // Proc. Intern. Conf. on Engineering and Applied Sci. Optimization. Athens: NTUA, 2014. P. 1001–1014.
3. Третьяков А. В. Автоматизация составления расписаний для систем реального времени // Труды ИСП РАН. 2012. **22**. С. 375–400.
4. Zhou T., Xiong H., Zhang Z. Hierarchical resource allocation for integrated modular avionics systems // J. Systems Engineering and Electronics. 2011. **22**. N 5. P. 780–787.
5. Lu H. et al. Scheduling based on interruption analysis and PSO for strictly periodic and preemptive partitions in integrated modular avionics // IEEE Access. 2018. **6**. P. 13523–13540.
6. Annighofer B., Thielecke F. A systems architecting framework for optimal distributed integrated modular avionics architectures // CEAS Aeronautical J. 2015. **6**. N 3. P. 485–496.
7. Sghairi M. et al. Challenges in building fault-tolerant flight control system for a civil aircraft // IAENG Intern. J. Computer Sci. 2008. **35**. N 4. P. 1–5.
8. Wang D. et al. Studying on ARINC653 partition run-time scheduling and simulation // WASET Intern. J. Computer, Electrical, Automation, Control and Information Engineering. 2012. **6**. P. 1482–1486.
9. Zhou Q. et al. TS-preemption threshold and priority optimization for the process scheduling in integrated modular avionics // Bio-Inspired Computing: Theories and Applications. Communications in Computer and Information Sci. Vol. 791. Singapore: Springer, 2017. P. 9–23.
10. Chen J., Du C., Han P. Scheduling independent partitions in integrated modular avionics systems // PloS One. 2016. **11**. N 12. P. 1–18.
11. Singhoff F. et al. Investigating the usability of real-time scheduling theory with the Cheddar project // Real-Time Systems. 2009. **43**. N 3. P. 259–295.
12. Craveiro J. P., Silveira R. O., Rufino J. HSSim: an extensible interoperable object-oriented n-level hierarchical scheduling simulator // Proc. 3d Intern. Workshop on Analysis Tools and Methodologies for Embedded and Real-time Systems (WATERS). Pisa: EUROMICRO Technical Committee on Real-Time Systems, 2012. P. 9–14.
13. Смелянский Р. Л. Модель функционирования распределенной вычислительной системы с временем // Программирование. 2013. **39**. № 5. P. 22–34.
14. Antonenko V. A. et al. DYANA: HLA-based distributed real-time embedded systems simulation tool // Proc. 2013 Winter Simulation Conf. Washington: IEEE Press, 2013. P. 4012–4013.
15. Буздалов Д. В. и др. Инструментальные средства проектирования систем интегрированной модульной авионики // Труды ИСП РАН. 2014. **26**. С. 201–230.
16. Wang L. et al. A tool for IMA system configuration verification and case study // Advances in Intelligent Systems and Interactive Applications. Advances in Intelligent Systems and Computing. Vol 686. Cham: Springer, 2017. P. 254–260.
17. Dissaux P. et al. The SMART project: multi-agent scheduling simulation of real-time architectures // Embedded Real Time Software and Systems. Toulouse: SEE, 2014. P. 1–9.
18. Cassez F., Larsen K. The impressive power of stopwatches // CONCUR 2000 — Concurrency Theory. Lecture Notes in Computer Science. Vol. 1877. Berlin: Springer, 2000. P. 138–152.
19. Glonina A. B., Bahmurov A. G. Stopwatch automata-based model for efficient schedulability analysis of modular computer systems // Parallel Computing Technologies (PaCT). Lecture Notes in Computer Science. Vol. 10421. Cham: Springer, 2017. P. 289–300.

20. G l o n i n a A. B., B a l a s h o v V. V. On the correctness of real-time modular computer systems modeling with stopwatch automata networks // Automatic Control and Computer Sciences. 2018. **52**. N 7. P. 817–827.
21. A n d r e E. Observer patterns for real-time systems // Proc. 2013 18th Intern. Conf. on Engineering of Complex Computer Systems. Singapore: IEEE Press, 2013. P. 125–134.
22. UPPAAL Home. URL: <http://uppaal.org/>
23. Cheetah3, the Python-powered template engine. URL: <http://cheetahtemplate.org/>
24. B a l a s h o v V. V. et al. Tool system and algorithms for scheduling of computations in integrated modular onboard embedded systems // IFAC Proc. Volumes. 2016. **49**. N 25. P. 505–510.
25. B a l a s h o v V. V., A n t i p i n a E. A. Distribution of workload in IMA systems by solving a modified multiple knapsack problem // Proc. 6th Intern. Conf. on Engineering Optimization. Cham: Springer, 2018. P. 1166–1177.

Поступила в редакцию 25.12.19

После доработки 08.04.20

Принята к публикации 08.04.20